

Propriedades, exemplos e confiança nos testes: uma resenha crítica com aplicação a números romanos

Suzan Stockey Pereira
Verificação e Validação de Software
PUCRS – Escola Politécnica

Repositório: <https://github.com/SuzanStockey/testes-numeros-romanos>

1 Introdução

Esta resenha compara três textos: o método de seleção por categorias de Ostrand e Balcer [1], a proposta de QuickCheck de Claessen e Hughes [2] e o estudo industrial de Goldstein et al. [3]. A aplicação é o *Roman Numerals Helper*, do Codewars [4]. O objetivo é discutir como seleção por partições, valores-limite e propriedades contribuem para testar o mesmo sistema, sem presumir superioridade de uma técnica.

O sistema usa Java 17, JUnit 5.13.4, jqwik 1.9.3 e Maven 3.9.11. As operações `toRoman(int)` e `fromRoman(String)` convertem inteiros de 1 a 3999 e romanos canônicos. O contrato inclui a rejeição de entradas inválidas com `IllegalArgumentException`, sem normalização: uma decisão própria, não exigida pelo Codewars. O artigo do E5 é QuickCheck; o registro de busca está em `pesquisa-e5.md`.

2 Referencial e comparação crítica

2.1 QuickCheck: propriedades como especificações executáveis

Claessen e Hughes [2] apresentam uma ferramenta leve para Haskell que combina propriedades executáveis, geração aleatória e apresentação de contraexemplos. Geradores personalizados, pré-condições e classificação dos dados permitem adequar a exploração ao domínio. Uma propriedade expressa uma expectativa geral; sua execução aleatória verifica apenas as amostras produzidas, sem constituir prova de correção.

A contribuição é sustentada por implementação, exemplos e relatos de aplicação, e não por um experimento comparativo padronizado. Seu ponto forte é tornar explícita a relação entre especificação, oráculo e geração. No exemplo de unificação, mais de 95% dos casos aceitos eram triviais; um gerador específico reduziu essa proporção para aproximadamente 20–25%. Esse resultado é daquele exemplo e mostra por que contar execuções não basta para julgar os testes. O artigo também relata falhas na própria especificação e atribui ao usuário a avaliação da distribuição e da suficiência da campanha.

Os casos apresentados demonstram viabilidade, mas não estimam ganhos universais de produtividade ou detecção. Além disso, propriedades e geradores são código sujeito a erros. A redução de contraexemplos aparece em um relato de extensão na seção sobre impressão de documentos; não se deve atribuir à implementação original toda a infraestrutura dos frameworks atuais. Neste trabalho, a redução é realizada pelo jqwik.

2.2 PBT na prática: benefícios e dificuldades persistentes

Goldstein et al. [3] investigam uma cultura madura de PBT na Jane Street, predominantemente em OCaml. O estudo qualitativo recrutou 31 pessoas e realizou 30 entrevistas, uma delas conjunta. A

análise temática examina benefícios, escrita de propriedades, geradores, depuração e interpretação de testes aprovados. As frequências relatadas têm entrevistas como unidade e não representam taxas de eficácia experimental.

Participantes descrevem descoberta de casos inesperados, documentação e maior compreensão do comportamento. Relações de ida e volta e comparação com implementações de referência ajudam quando uma especificação útil já está disponível. Contudo, construir dados válidos e obter uma distribuição relevante continua trabalhoso. O estudo também destaca a necessidade de avaliar o próprio teste: entre as práticas relatadas estão inspeção de entradas e introdução de defeitos. Exemplos e propriedades têm utilidades complementares para comunicar expectativas e explorar comportamentos.

O ponto forte é observar usuários experientes ao longo do processo de desenvolvimento, incluindo dificuldades que persistem após a adoção. A limitação principal é o contexto de uma empresa e de um ecossistema específico, com participantes experientes e possíveis vieses de seleção e lembrança. Relatos de confiança e defeitos encontrados não substituem medições controladas; tampouco estabelecem como iniciantes em Java se comportariam.

2.3 Category-Partition: seleção a partir da especificação

Ostrand e Balcer [1] apresentam *The Category-Partition Method for Specifying and Generating Functional Tests*: a especificação é organizada em categorias, escolhas e restrições, das quais se obtêm descrições de testes e entradas concretas. Seu ponto forte é explicitar a seleção e evitar combinações incompatíveis. O artigo apresenta a ferramenta TSL, exemplos e experiência de aplicação, sem comparação controlada de superioridade. Categorias e resultados esperados ainda exigem julgamento do testador.

2.4 Comparação e posição adotada

Os textos têm enfoques complementares: seleção por categorias, propriedades executáveis e experiência industrial. A seleção manual dos exemplos se relaciona aos conceitos de Ostrand e Balcer; não foram usados TSL nem a geração completa de suas combinações. QuickCheck e o estudo de 2024 fundamentam os geradores e as verificações de PBT.

Adotou-se a combinação de exemplos conhecidos e propriedades com referências independentes. A ligação entre literatura e aplicação fica explícita: Ostrand e Balcer sustentam a organização por partições; os dois artigos de PBT sustentam relações e geração; a AVL verifica extremos e transições derivados do contrato. Não se presume que a combinação detecte mais defeitos.

3 Aplicação das técnicas ao conversor

Particionamento de equivalência (PE). Os 12 casos representativos foram relacionados às categorias e escolhas de Ostrand e Balcer [1]: subtração (nenhuma, uma, duas/três posições) e magnitude (quatro faixas) são eixos de partição; composição é um checklist com sobreposição. Não são 12 classes disjuntas, nem foi testado o produto completo das categorias. Os pares usam resultados literais; 944 = CMXLIV exercita três posições subtrativas.

Análise de valores-limite (AVL). A seleção de entradas válidas contém 25 números: extremos e vizinhos de 4, 9, 40, 90, 400 e 900. Após consolidar três pares compartilhados com PE, ficaram 34 pares nos dois sentidos: 68 comparações e um teste de sequência, totalizando 69 itens. A validação do contrato verifica também 0, 1, 3999 e 4000: os externos devem ser rejeitados, os internos convertidos corretamente.

Testes baseados em propriedades. Seja $N = \{1, \dots, 3999\}$, E o codificador, D o decodificador, C a verificação de formato canônico e O um modelo independente de codificação. Foram implementadas seis propriedades:

ID	Verificação
P01	$D(E(n)) = n$, após conferir $C(E(n))$
P02	$E(D(r)) = r$, após conferir $D(r) \in N$
P03	$C(E(n))$
P04	$D(r) \in N$
P05	$E(n) = O(n)$
P06	$D(r) = n$, com par independente (n, r)

O codificador de produção usa valores decrescentes, e o decodificador identifica subtrações entre símbolos vizinhos. O modelo O usa tabelas por posição decimal; C combina expressão regular integral e verificação de não vazio. Os pares (n, r) são construídos pelo modelo, evitando usar E como referência para testar D . A redução diminui n e reconstrói r , preservando validade e correspondência do par. Os oráculos foram conferidos por 35 testes de infraestrutura, separados dos testes do sistema.

Gerar diretamente dados válidos evita descartes por pré-condição. A distribuição mistura 80% de inteiros uniformes de N e 20% dos 25 valores-limite. Coletaram-se categorias de magnitude, presença de subtração e centros/extremos. Usaram-se as sementes 42, 2024, 2026, 3999 e 104729, até 1000 avaliações por propriedade/semente, com modo explicitamente aleatório. A injeção automática de casos especiais e o banco de falhas foram desabilitados. Sementes iguais reutilizam sequências; avaliações não equivalem a entradas distintas ou independentes.

4 Resultados e discussão

Na base da campanha de defeitos (revisão 78399cf), 69 exemplos, 35 itens de infraestrutura e 30 execuções de propriedades passaram: 134 itens e 30.000 avaliações primárias, sem descartes. A enumeração complementar verificou 7998 comparações direcionais separadamente. A matriz abaixo corresponde ao snapshot dessa campanha.

Para avaliar os testes, introduziu-se um defeito por vez, sempre a partir da mesma base. A tabela apresenta falhas de asserção, e não quantidade de defeitos ou entradas diferentes. O grupo combinado reúne somente exemplos e PBT.

Variante	Defeito	Exemplos	PBT	Combinação
DF-01	Emitir IIII em vez de IV	3/69	20/30	23/99
DF-02	Ignorar subtração na leitura	21/69	20/30	41/99
DF-03	Omitir unidades com zero interno	5/69	15/30	20/99
DF-04	Codificar 3999 como 3998	1/69	15/30	16/99

Os três grupos detectaram 4/4 variantes; o PBT detectou cada uma nas cinco sementes, sem ganho da combinação. Os checks primários foram 10.212, 10.116, 15.060 e 16.638, excluídas as avaliações de redução. Na validação do contrato, 45 itens verificam números fora do domínio, textos inválidos e fronteiras válidas; duas propriedades acrescentam 2000 checks de rejeição. A suíte atual aprovou 181 itens e 32.000 checks, sem descartes; a enumeração de 7998 comparações também passou. A campanha de defeitos não foi repetida nessa versão.

4.1 O significado das propriedades e dos contraexemplos

P01 e P02 detectaram as quatro variantes nas cinco sementes. P03 detectou apenas DF-01: resultados de DF-03 e DF-04 continuaram canônicos, embora numericamente errados. P04 detectou DF-02 quando a soma ultrapassou o domínio, mas aprovou os defeitos exclusivos do codificador. P05 detectou DF-01, DF-03 e DF-04; P06 detectou DF-02. Isso evidencia que formato e intervalo isolados não caracterizam a semântica completa.

Um codificador que emita IIII para 4 e um decodificador que o leia como 4 podem manter a inversão e violar o formato. A guarda de canonicidade em P01 impede essa falsa confiança. O cenário não é uma quinta variante executada; RP-03 exige a rejeição de IIII.

Para P01 e semente 42, o jqwik reduziu as entradas de DF-01 de 2504 para 4, de DF-02 de 999 para 4 e de DF-03 de 2504 para 901. Em DF-03, 901 perde a unidade por conter zero nas dezenas. Em DF-04, o contraexemplo final permaneceu 3999. São resultados do redutor, sem garantia de mínimo global. Os logs guardam sementes, amostras e mensagens para reprodução e diagnóstico.

4.2 Dificuldades e ameaças à validade

As decisões exigiram delimitar o contrato, distinguir testes de avaliações e preservar invariantes na redução. A validação rejeita `null`, vazio, espaços, minúsculas e formas como IIII ou IC. O decodificador compara a entrada com a recodificação do valor, criando dependência do codificador; testes com modelo posicional independente e enumeração reduzem esse risco. Dois geradores de rejeição cobrem inteiros externos e símbolo desconhecido, não todas as strings inválidas. O código e os testes são Java.

Os defeitos são artificiais e exercitam requisitos já cobertos pelos exemplos escolhidos. O domínio pequeno e a ênfase nos limites favorecem a detecção, especialmente de DF-04. Portanto, não há evidência de superioridade do PBT, ganho marginal da combinação ou vantagem de custo. Cinco sementes não são cinco estudos independentes. A validação dos oráculos e sua diferença algorítmica em relação ao sistema reduzem, mas não eliminam, erros compartilhados. A enumeração auxiliar confirma o efeito das variantes e não integra a métrica de detecção. Código e testes foram produzidos no mesmo processo, sem revisão externa independente, o que também limita a validade.

5 Conclusão

Exemplos e PBT detectaram os mesmos quatro defeitos na campanha de defeitos; formato ou intervalo isolados deixaram erros semânticos passar. O contrato inclui rejeição explícita e testes de classes inválidas, mantendo as conversões válidas: 181 itens aprovados. As campanhas têm versões e evidências identificadas, sem extrapolação à eficácia industrial.

Uso de IA

A IA foi utilizada, sob diretrizes, acompanhamento e revisão da autora, como apoio à pesquisa e análise dos artigos, à elaboração de código e testes e à execução dos experimentos. Também auxiliou na revisão e refatoração do projeto, na inclusão de casos de entradas inválidas e valores-limite e nos ajustes da documentação. A autora revisou as alterações e permanece responsável pelas decisões e pela entrega.

Referências

- [1] T. J. Ostrand and M. J. Balcer, “The Category-Partition Method for Specifying and Generating Functional Tests,” *Communications of the ACM*, vol. 31, no. 6, pp. 676–686, 1988. doi: 10.1145/62959.62964.
- [2] K. Claessen and J. Hughes, “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs,” in *Proc. ICFP*, 2000, pp. 268–279. doi: 10.1145/351240.351266.
- [3] H. Goldstein, J. W. Cutler, D. Dickstein, B. C. Pierce, and A. Head, “Property-Based Testing in Practice,” in *Proc. ICSE*, 2024, art. 187, pp. 1–13. doi: 10.1145/3597503.3639581.
- [4] Codewars, “Roman Numerals Helper.” [Online]. Available: <https://www.codewars.com/kata/51b66044bce5799a7f000003>. Acesso em: 7 out. 2026.